Part 1
일주일 만에 끝내는 파이썬 기초

1장 Hello Python!,
2장 데이터 입력하고 결과 출력하기
3장 변수로 데이터 저장하기
4장 조건에 따라 해야 할 일이 다를 때: 조건문
5장 같은 일을 여러 번 반복해야 할 때: 반복문
6장 여러 데이터를 한 번에 묶어 표현하기: 리스트
7장 문자 기준으로 데이터 묶기: 딕셔너리

1장
Hello, Python!

많은 학생에게 파이썬 교육을 의무로 시행하다는 대학교가 늘고 있습니다. 왜 학생이 파이썬을 배우라는 지에는 궁금하고 있습니다. 온라인에서 문과생인데 파이썬을 배울 수 있다는 글로를 검색할 수 있습니다. 파이썬이 도대체 무엇이길래 이렇게 많은 사람들이 배우라는 것인지, 왜 배울 수 있는 것인지 궁금해 하는 분들이 많을 것입니다.

먼저 이 책에 질문에 대답하고 답하겠습니다. 컴퓨터 사고력을 강조하는 이유부터 코드라는 것이 무엇인지 파이썬을 배울 때 무엇을 얻어갈 수 있는지 등에 대해 설명하는 것부터 시작하겠습니다. 그리고 실제로 간단한 프로그램을 작성해 보면서 프로그램을 작성하는 것이 어떤 것인지 경험해 보는 것으로 마무리하겠습니다.

1.1 컴퓨터 사고력과 데이터를 다루는 능력

최근 코딩 교육, 컴퓨터 사고력 등의 키워드가 많이 등장하고 있습니다. 컴퓨터 사고력이 무엇인지 궁금하신 분들이 많을 것입니다. 컴퓨터 사고력은 컴퓨터 과학에서 사용하는 문제 해결 방법을 의미합니다. 컴퓨터 과학에서 사용하는 문제 해결 방법은 크게 세 가지로 나눌 수 있습니다. 첫 번째는 문제를 분석하는 것입니다. 문제를 분석하는 과정에서 문제를 해결하기 위해 필요한 데이터가 무엇인지 파악하고 데이터를 어떻게 처리해야 하는지 고민하게 됩니다. 두 번째는 문제를 해결하기 위한 알고리즘을 설계하는 것입니다. 알고리즘은 문제를 해결하기 위한 절차를 의미합니다. 마지막으로 세 번째는 설계한 알고리즘을 프로그램으로 구현하는 것입니다. 프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다. 이 책에서는 파이썬 언어를 사용하여 프로그램을 작성하는 방법을 배우게 됩니다.

1.1.1 컴퓨터 사고력

컴퓨터 사고력은 컴퓨터 과학에서 사용하는 문제 해결 방법을 의미합니다. 컴퓨터 사고력은 문제를 분석하는 것, 알고리즘을 설계하는 것, 프로그램을 구현하는 것으로 나눌 수 있습니다.

1.1.2 데이터를 다루는 능력

컴퓨터 사고력에서 데이터를 다루는 능력은 매우 중요합니다. 데이터를 다루는 능력은 데이터를 수집하는 것, 데이터를 분석하는 것, 데이터를 시각화하는 것으로 나눌 수 있습니다. 데이터를 수집하는 것은 데이터를 분석하기 위해 필요한 데이터를 수집하는 것입니다. 데이터를 분석하는 것은 데이터를 분석하여 필요한 정보를 추출하는 것입니다. 데이터를 시각화하는 것은 데이터를 그래프로 표현하는 것입니다.

1.1.3 알고리즘

알고리즘은 문제를 해결하기 위한 절차를 의미합니다. 알고리즘은 문제를 해결하기 위한 절차를 의미합니다. 알고리즘은 문제를 해결하기 위한 절차를 의미합니다. 알고리즘은 문제를 해결하기 위한 절차를 의미합니다. 알고리즘은 문제를 해결하기 위한 절차를 의미합니다. 알고리즘은 문제를 해결하기 위한 절차를 의미합니다. 알고리즘은 문제를 해결하기 위한 절차를 의미합니다. 알고리즘은 문제를 해결하기 위한 절차를 의미합니다. 알고리즘은 문제를 해결하기 위한 절차를 의미합니다. 알고리즘은 문제를 해결하기 위한 절차를 의미합니다.

1.1.4 프로그램

프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다. 프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다. 프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다. 프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다. 프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다. 프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다. 프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다. 프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다. 프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다. 프로그램은 컴퓨터가 이해할 수 있는 언어로 작성해야 합니다.

1.1

프로그램과 프로그래밍

1.1.1 프로그램

프로그램은 우리 생활과 떼려야 뗄 수 없는 단어입니다. 아침에 알람을 울리는 스마트폰 알람 앱도 프로그램이고, 다음 버스가 언제 오는지 알려 주는 버스 정보 시스템도 프로그램입니다. 다른 사람과 대화할 수 있는 메신저도 프로그램이고요.

프로그램(program)이란 어떤 문제를 해결하기 위해 컴퓨터가 실행해야 하는 명령어의 집합체입니다. 명령은 다른 사람에게 무엇을 하게 하는 것입니다. 프로그램에서 명령은 사람이 컴퓨터에 어떤 일을 시키는 것을 의미합니다.

메신저 프로그램을 생각해 봅시다. 메신저 프로그램의 목적은 다른 사람과 대화하는 것입니다. 상대방에게 텍스트를 보낼 때는 [전송], 사진을 보낼 때는 [사진] 버튼을 누릅니다. [전송] 버튼은 입력한 내용을 상대방에게 전달하라는 명령어고, [사진] 버튼은 내 기기에 저장된 사진 중 하나를 선택하라는 명령어입니다.

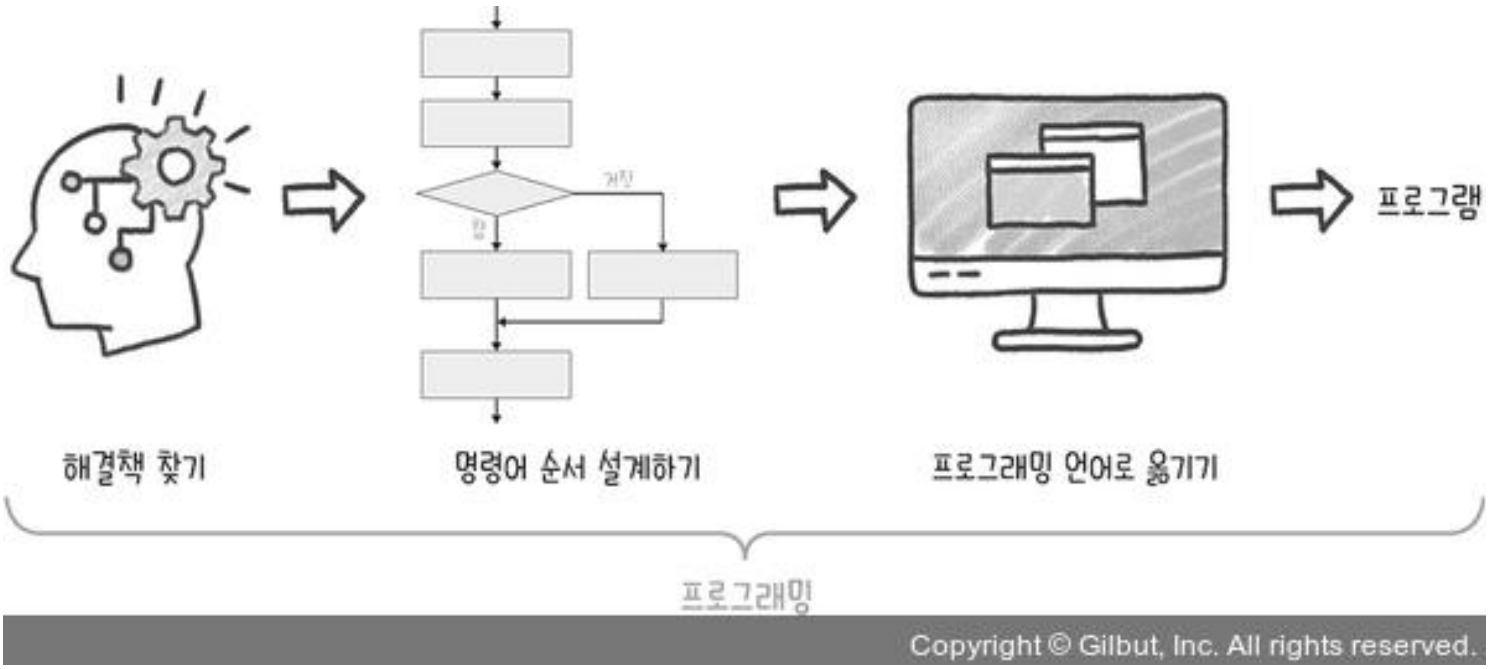


▲ 그림 1-1 메신저 프로그램에서 명령어

이처럼 메신저에서도 사람이 시키는 작업을 수행하려면 그에 맞는 여러 명령어가 필요합니다. 그리고 이런 명령어가 모여 프로그램이 되는 것입니다.

1.1.2 프로그래밍

그렇다면 프로그래밍은 무엇일까요? 프로그래밍(programming)은 컴퓨터 프로그램을 만들 때 필요한 모든 행위를 포함하는 개념입니다. 프로그래밍은 프로그램을 만드는 데 필요한 명령어의 순서를 설계하고, 프로그래밍 언어로 옮겨 실제 프로그램으로 만드는 모든 과정을 포함합니다.



Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 1-2 프로그래밍의 의미

그렇다면 코딩과 프로그래밍의 차이는 무엇일까요? 코딩(coding)이란 소스 코드를 작성하는 행위를 말합니다. 이때 소스 코드(source code)는 프로그래밍 언어로 작성된 텍스트 파일을 의미하며, 이 소스 코드를 실행하면 프로그램이 작동합니다.

그림 1-2에서 프로그래밍 언어로 옮기는 단계가 바로 코딩에 해당합니다. 따라서 코딩은 프로그래밍에 포함되는 개념이죠. 실제로는 코딩과 프로그래밍을 엄격히 구분해 사용하지는 않지만, 차이를 알아 둘 필요는 있습니다.

1.1.3 프로그래밍의 가치

예전에는 프로그래밍을 개발자만의 영역으로 여겼습니다. 하지만 앞에서 이야기한 것처럼, 비전공자도 직장인도 심지어 초·중·고등학생도 프로그램을 배우는 추세입니다. 이제 프로그래밍 역량은 모든 사람에게 요구됩니다. 프로그래밍에는 어떤 가치가 있길래 이런 일이 벌어지는 걸까요?

프로그램은 대부분 현실의 문제를 해결하기 위한 목적으로 만듭니다. 예를 들어 버스가 언제 올지 승객들이 무작정 기다리는 것이 문제라면 버스 도착시간을 알 수 있도록 버스 정보 시스템을 만드는 것입니다. 또한, 처음 가는 장소라 길을 잘 모르는 것이 문제라면 쉽게 찾아가도록 길 찾기 프로그램을 만드는 것이죠. 이처럼 문제가 있으면 이에 따른 해결책을 찾습니다.

이 해결책이 곧 아이디어입니다. 아이디어를 실현하기 위해 필요한 명령을 도출하고, 순서를 설계하고, 실제 코드로 옮기면 프로그램이 됩니다. 즉, 프로그램은 아이디어를 구현한 결과물이죠. 아이디어는 현대 교육에서 강조하는 창의성과 직결됩니다. 아이디어를 프로그램으로 만들기 위해 다양한 노력을 기울이는 과정에서 창의성을 키우게 됩니다. 프로그래밍은 창의성을 기르고 아이디어를 현실화하는 효과적인 수단이기 때문에 프로그래밍 교육이 확대될 수밖에 없습니다. 프로그래밍 교육이 확대되면 누구나 창의적인 아이디어를 실제 프로그램으로 구현할 수 있습니다.

현대 사회에서는 대부분 아침에 일어나서 잠들 때까지 수많은 프로그램을 사용합니다. 그리고 새로운 프로그램이 끊임없이 등장하며 세상을 변화시킵니다. 따라서 개발자가 아니더라도 자신의 분야에서 특별한 해결책, 혁신적인 아이디어를 제시할 수 있으려면 프로그래밍을 알고 있는 게 당연히 유리합니다.

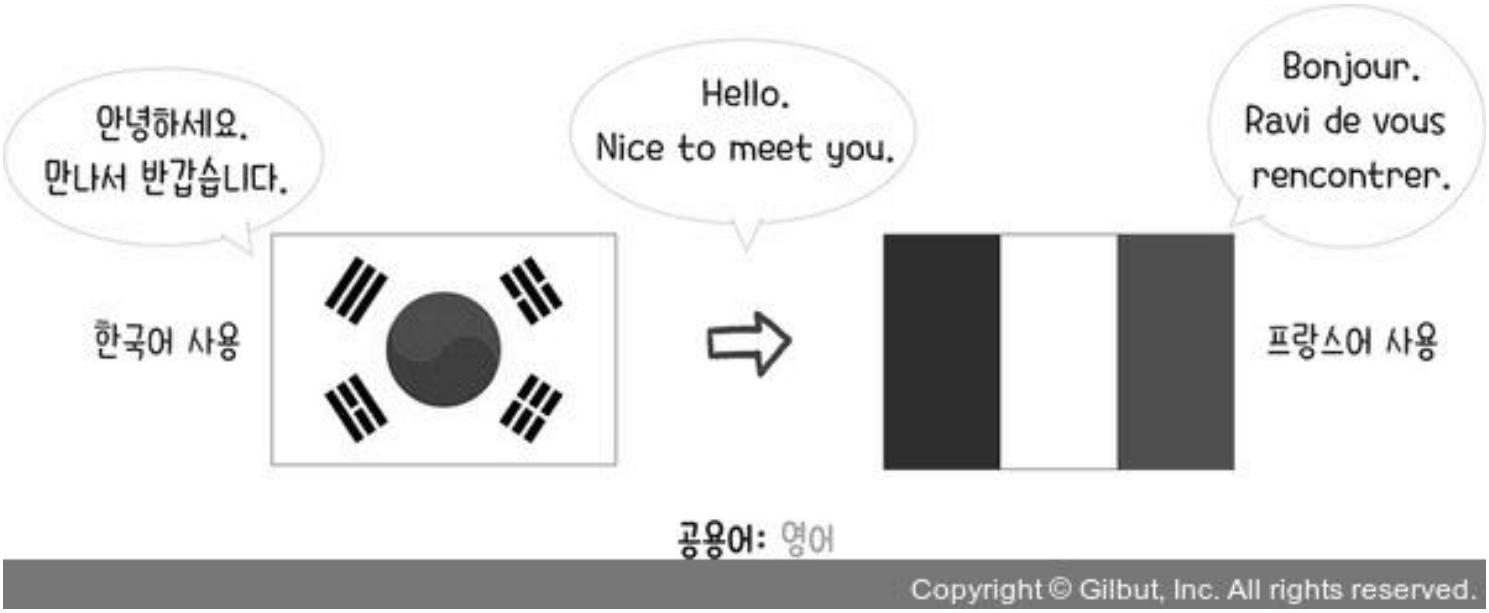
1.2

파이썬 프로그래밍을 시작하기 전에

1.2.1 파이썬이란

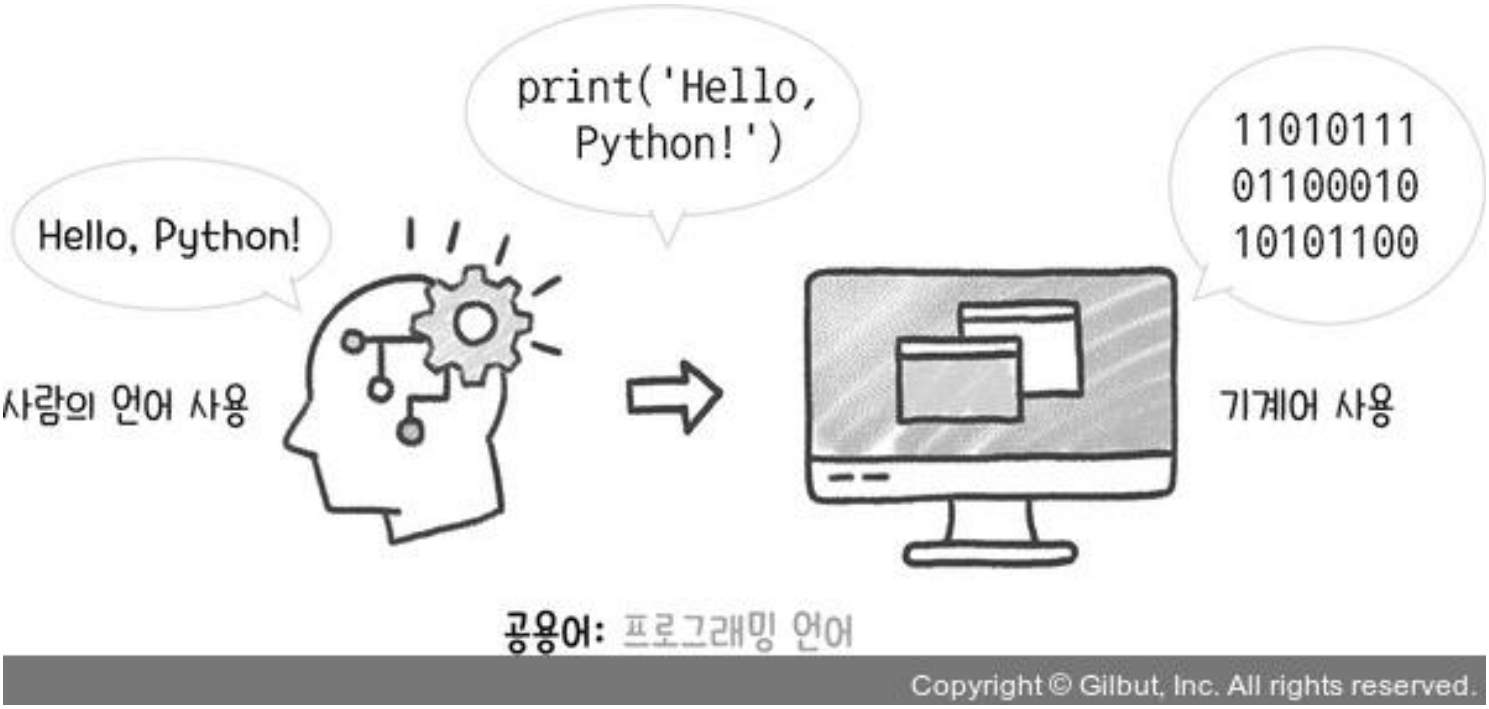
프로그램과 프로그래밍의 정의 그리고 프로그래밍의 가치에 대해서 이해했을 겁니다. 지금부터는 이 책에서 배울 파이썬이 무엇인지 살펴봅시다.

파이썬은 프로그래밍 언어 중 하나입니다. 프로그래밍 언어란 사람과 컴퓨터의 중간자 역할을 하는 언어입니다. 예를 들어 봅시다. 한국인과 프랑스인이 대화하려고 합니다. 그런데 양측 모두 자신의 언어로만 말해서 소통하지 못하고 있습니다. 어떻게 해야 서로 대화할 수 있을까요? 한국인과 프랑스인이 둘 다 알고 있는 제3의 언어를 사용하면 됩니다. 그 언어가 영어라고 해 봅시다. 한국인이 프랑스인에게 말할 때는 한국어를 영어로 바꿔 말하고, 프랑스인은 영어를 프랑스어로 바꿔 이해합니다. 반대로 프랑스인이 말할 때는 프랑스어를 영어로 바꿔 말하고, 한국인은 영어를 한국어로 바꿔 이해합니다.



▲ 그림 1-3 한국인과 프랑스인의 대화

사람과 컴퓨터의 관계도 이와 비슷합니다. 컴퓨터는 0과 1만으로 이루어진 언어를 사용합니다. 사람이 이해하기 매우 어렵죠. 그래서 둘 다 이해할 수 있는 제3의 언어를 사용하는데, 이 언어가 바로 프로그래밍 언어입니다. 사람은 프로그래밍 언어로 명령을 내립니다. 그럼 컴퓨터는 프로그래밍 언어를 0과 1로 변경해 이해하고 명령을 수행합니다. 이처럼 사람과 컴퓨터가 대화할 수 있도록 중간 공용어 역할을 하는 것이 프로그래밍 언어입니다.



▲ 그림 1-4 컴퓨터가 이해하는 프로그래밍 언어

프로그래밍 언어는 종류가 많습니다. 포트란(1954년), 코볼(1959년), 베이직(1964년) 등의 초창기 언어부터 현재도 많이 사용하는 C(1972년), C++(1983년) 등이 있죠. 이런 언어들과 비교하면 파이썬은 비교적 최근에 등장한 언어입니다. 파이썬(Python)은 1991년에 귀도 반 로섬(Guido van Rossum)이 발표했습니다.

파이썬은 다른 프로그래밍 언어와 구별되는 강력한 장점이 있는데, 두 가지만 살펴보겠습니다.

1. 쉽고 직관적인 문법

파이썬은 다른 프로그래밍 언어보다 문법이 쉽습니다. 다소 어렵게 느껴질 수 있는 부분을 숨기거나 단순화해 배우기 쉽습니다. 이런 이유로 프로그래밍을 처음 배우는 사람(입문자)에게 많이 추천합니다.

2. 강력하고 풍부한 라이브러리

라이브러리는 확장 기능이라고 이해하면 쉽습니다(나중에 자세히 배웁니다). 라이브러리를 작성하는 파이썬 프로그램에 포함하면 다양한 확장 기능을 사용할 수 있습니다. 대표적으로 데이터를 시각화할 수 있는 `matplotlib`(맷플롯립), 다양한 머신러닝 모델을 지원하는 `scikit-learn`(사이킷런), 딥러닝을 구현할 수 있게 하는 `tensorflow`(텐서플로) 라이브러리 등을 제공합니다.

쉽고 직관적이며 기능이 풍부하므로 프로그래밍 입문자도, 능숙한 개발자도 파이썬을 배우는 추세입니다. 우리도 한번 파이썬을 배워 봅시다.

1.2.2 파이썬으로 프로그래밍하기 위한 준비 과정

그럼 이제부터 파이썬으로 프로그래밍하도록 준비해 봅시다.

에디터 선택하기

가장 먼저 파이썬으로 코드를 작성할 수 있는 도구가 필요한데, 이를 통합 개발 환경(IDE) 또는 간단하게 에디터라고 합니다. 파이썬을 지원하는 통합 개발 환경은 종류가 굉장히 많습니다.

이 책에서는 그중에서 두 가지 에디터를 사용합니다. 구글 코랩(Google Colab, 이하 코랩)은 표 1-1에서 알 수 있듯이 설치할 필요가 없어서 사용하기 편리합니다. 8장부터는 데이터 분석을 진행하는 데 편한 주피터 노트북을 사용합니다. 처음부터 주피터 노트북을 사용하거나 파이썬에 기본으로 포함된 파이썬 IDLE을 사용해도 됩니다. 본인에게 맞는 에디터를 선택해 진행하세요.

▼ 표 1-1 코랩과 주피터 노트북 비교

구분	코랩	주피터 노트북
사용 방식	설치할 필요 없이 구글 계정으로 로그인한 후 사용하는 방식	최초 1회 설치한 후 설치된 주피터 노트북을 실행해 사용하는 방식
인터넷 연결	항상 인터넷에 연결된 상태여야 하고, 구글 계정이 필요함	인터넷 연결 필요 없음(브라우저에서 HTML 창이 열리나 온라인 연결이 필요하지 않음)
서버	구글에서 제공하는 클라우드 서버에 온라인으로 접속해 사용함	로컬 PC(주피터 노트북을 설치한 PC)를 서버로 사용함
파이썬 파일 저장	파일은 구글 드라이브의 Colab Notebooks 폴더에 저장되므로 인터넷만 연결된다면 언제 어디에서든 파일을 열어 확인할 수 있음	로컬 PC에 저장됨

코랩 사용하기

코랩은 온라인으로 파이썬 프로그래밍할 수 있는 도구로, 주피터 노트북과 기능, 구조, 사용 방법이 매우 비슷합니다. 하지만 구글 서버를 사용하기 때문에 인터넷에 꼭 연결되어 있어야 합니다.

코랩 접속하기

1 크롬 브라우저를 열고 구글(https://www.google.com)에 접속합니다. 오른쪽 위에 있는 [로그인] 버튼을 눌러 로그인하세요. 구글 계정이 없다면 먼저 회원가입을 합니다. [로그인] 버튼이 안 보인다면 구글 검색창에서 '로그인'을 검색해 보세요. 오른쪽에 [로그인] 버튼이 보일 겁니다. 또는 첫 번째 검색결과로 뜨는 '로그인 - Google 계정'을 클릭해도 됩니다.



▲ 그림 1-5 구글 로그인하기

2 로그인한 후 구글 검색창에 'colab'을 검색합니다. 첫 번째 검색결과인 'Google Colab'을 클릭합니다.



▲ 그림 1-6 코랩 검색하기

3 코랩 페이지가 열립니다.



▲ 그림 1-7 코랩 접속하기

코랩 사용하기

코랩에서 파일을 생성해 봅시다. 앞의 화면에서 [새 노트]를 클릭하거나 메뉴에서 [파일 → 새 노트]를 선택합니다. 새로운 페이지가 열리고 다음과 같은 화면이 보입니다.



▲ 그림 1-8 새 파일 만들기

열린 파일을 보면 상단에 Untitled0.ipynb이라고 적힌 부분이 있습니다. 이 부분을 클릭하면 파일명을 수정할 수 있습니다. 확장자(.ipynb)를 제외하고 나머지 부분에 원하는 이름을 입력한 후 Enter를 누르면 파일명이 바뀝니다. 코랩은 주피터 노트북과 다르게 변경 사항을 자동으로 저장합니다. 이런 특징 덕분에 갑자기 PC가 꺼지거나 정전되는 등의 예상치 못한 상황에서 자유롭습니다.

코랩의 실행 단위는 셀(cell)입니다. 셀은 하나의 코드 블럭을 의미합니다. 코랩에는 셀 유형이 두 가지인데, 코드를 입력하는 코드 셀과 텍스트를 입력하는 텍스트 셀입니다. 메뉴 아래에 있는 [+코드]를 클릭하면 코드 셀을, [+텍스트]를 클릭하면 텍스트 셀을 추가할 수 있습니다.

파일을 새로 만들면 기본 코드 셀 하나가 열려 있습니다. 코드 셀을 추가하고 싶을 때는 [+코드]를 클릭합니다. 코드 셀을 실행하는 아이콘은 코드 셀 왼쪽에 있습니다. Ctrl + Enter 또는 Shift + Enter를 눌러도 됩니다. Ctrl + Enter는 현재 셀을 실행하는 단축키고, Shift + Enter는 현재 셀을 실행한 후 다음 셀로 이동하는 단축키입니다. 코드 셀을 삭제하려면 셀을 클릭했을 때 보이는 오른쪽 메뉴에서 휴지통 아이콘을 클릭하면 됩니다.



▲ 그림 1-9 코드 셀 추가/실행/삭제하기

텍스트 셀은 주피터 노트북의 마크다운과 동일합니다. [+텍스트]를 클릭하면 에디터 메뉴와 함께 양 옆으로 분할된 형태로 셀이 나타나는데, 왼쪽에 텍스트를 입력하면 오른쪽에 입력 결과를 미리 보여 줍니다. 마우스를 다른 셀로 옮기거나 Shift + Enter 를 누르면 입력된 내용이 반영됩니다. 텍스트 셀을 삭제하는 방법은 코드 셀을 삭제하는 방법과 동일합니다.



▲ 그림 1-10 텍스트 셀 추가/삭제하기

이제 코랩의 설정을 조금 만져 봅시다. 파일의 오른쪽 상단에 톱니바퀴 모양의 설정 버튼(

)이 있습니다. 설정 버튼을 클릭한 후 두 번째 탭에 있는 [편집기]를 선택합니다. 여기서 코랩 편집기의 설정을 수정할 수 있습니다. 간단하게 2개만 수정해 봅시다. 들여쓰기를 2에서 4로 변경하고, [행 번호 표시]를 체크합니다. 들여쓰기의 의미는 4장 조건문에서 다룹니다. [행 번호 표시]를 체크하면 코드 셀에 줄 번호를 표시합니다.



▲ 그림 1-11 코랩 편집기 설정하기

이렇게 작성한 코랩 파일은 구글 드라이브에 자동으로 저장되기 때문에 인터넷에 접속할 수만 있다면 언제 어디서나 파일을 확인하거나 편집할 수 있습니다. 드라이브에 코랩 파일이 저장됐는지 확인해 봅시다. 구글 검색창에 '구글 드라이브'를 검색합니다. 첫 번째 검색결과를 클릭하면 구글 드라이브로 이동합니다. 또는 구글 초기화면에서 로그인 창 옆에 있는 구글 앱 버튼(

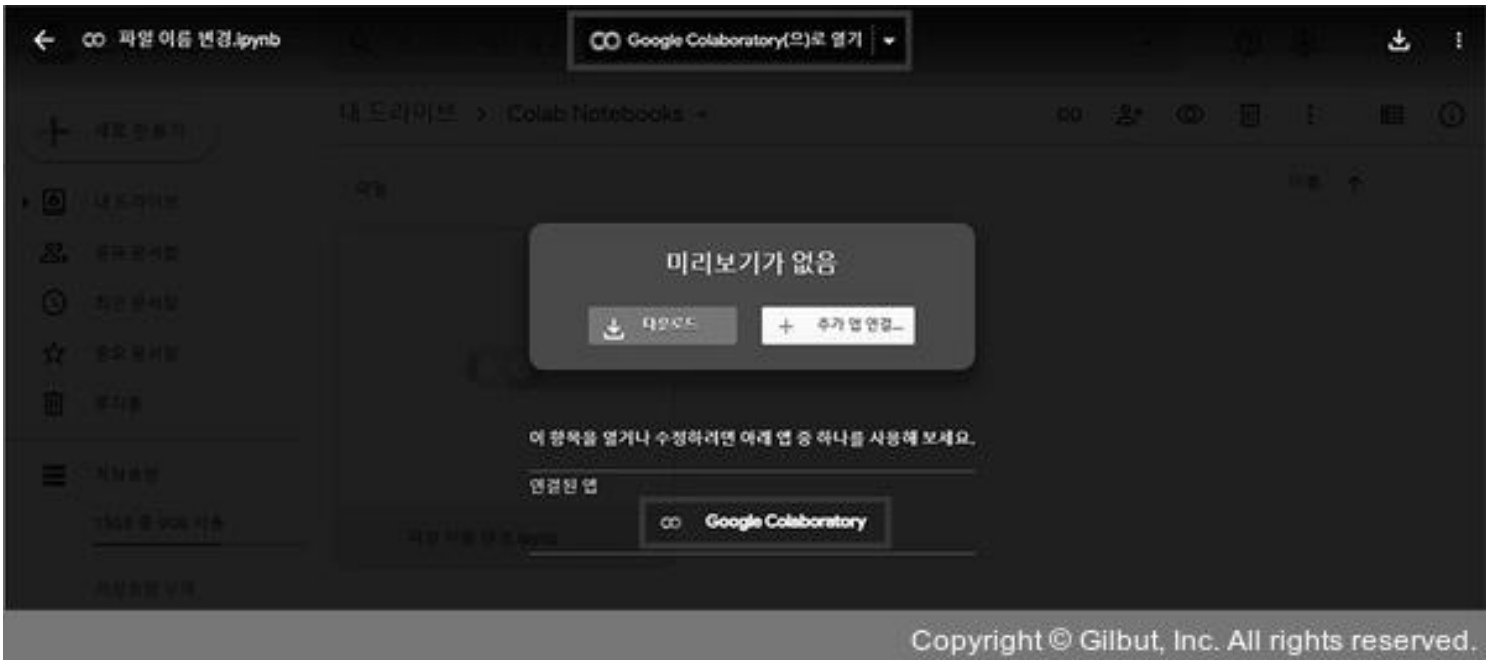
)을 클릭하면 구글 드라이브에 입장할 수 있습니다.

코랩을 처음 사용하면 구글 드라이브에 노란색의 Colab Notebooks 폴더가 자동으로 생성됩니다. 이 폴더를 클릭하면 앞에서 만든 파일을 확인할 수 있습니다.



▲ 그림 1-12 구글 드라이브에 저장된 코랩 파일 확인하기

구글 드라이브에서 코랩 파일을 클릭한 후, 상단의 [Google Colaboratory으로 열기]나 연결된 앱 [Google Colaboratory] 중 하나를 클릭하면 해당 파일을 코랩으로 다시 열 수 있습니다.



▲ 그림 1-13 구글 드라이브에서 코랩 파일 열기

주피터 노트북(아나콘다) 설치하기

프로그래밍하다 보면 라이브러리 설치가 까다롭고 귀찮은 경우가 많습니다. 이런 불편함을 제거해 간편하게 프로그래밍할 수 있게 돕는 것이 바로 아나콘다입니다. 아나콘다(Anaconda)는 파이썬과 함께 파이썬에서 자주 활용하는 주요 라이브러리를 설치해 줍니다. 주피터 노트북도 아나콘다를 설치하면 함께 설치됩니다.



▲ 그림 1-14 아나콘다에 포함된 라이브러리

아나콘다 설치하기

그럼 아나콘다를 설치해 보겠습니다. 그림을 보고 따라 하면 돼서 어렵지 않습니다.

1 브라우저에서 https://www.anaconda.com 사이트에 접속해 상단 메뉴에서 [Products → Individual Edition]을 선택합니다.



▲ 그림 1-15 아나콘다 사이트 접속하기

2 다운로드 페이지가 열리면 [Download] 버튼을 클릭합니다.



▲ 그림 1-16 아나콘다 설치 파일 내려받기

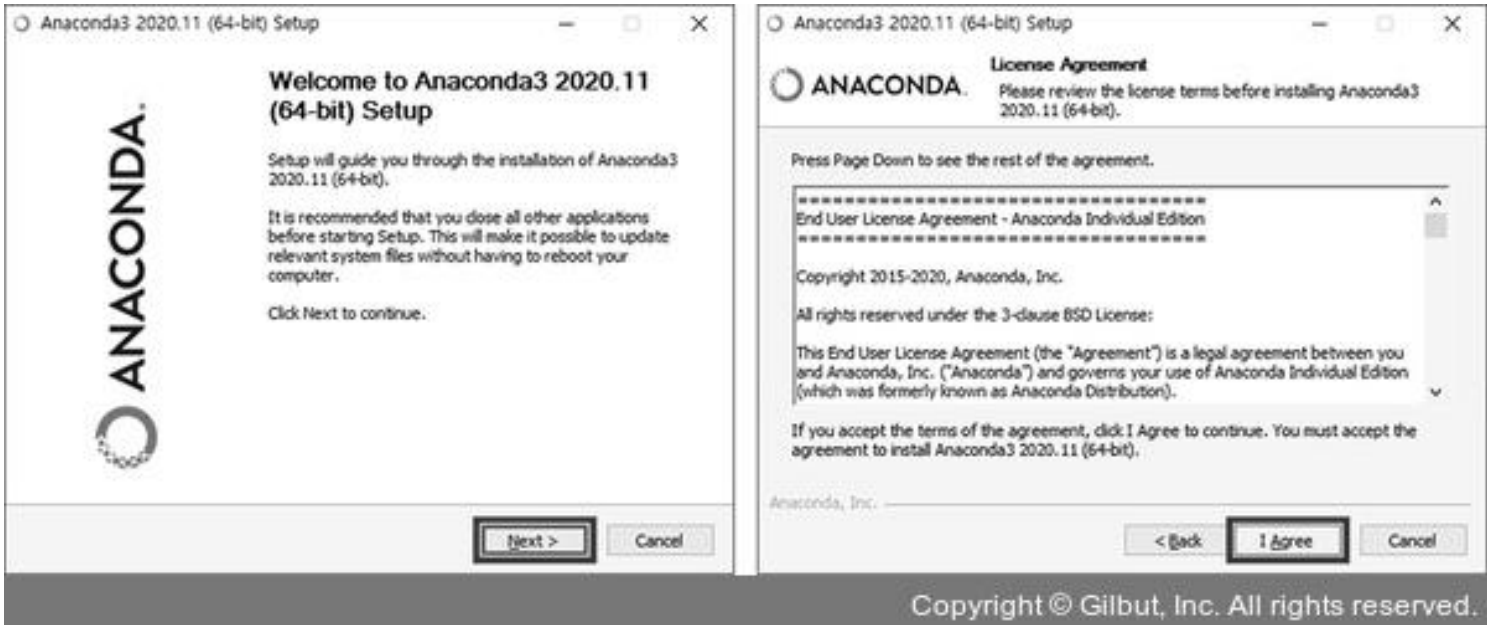
3 설치하려는 PC에 맞는 버전을 선택합니다. 운영 체제가 32bit라면 32-Bit Graphical installer를, 운영체제가 64bit라면 64-Bit Graphical installer를 선택합니다.

> TIP 현재 최신 아나콘다는 파이썬 3.8을 지원하므로 해당 버전을 내려받습니다. 이 책에 사용한 예제는 파이썬 버전과 거의 상관이 없으므로 최신 버전으로 내려받으면 됩니다.



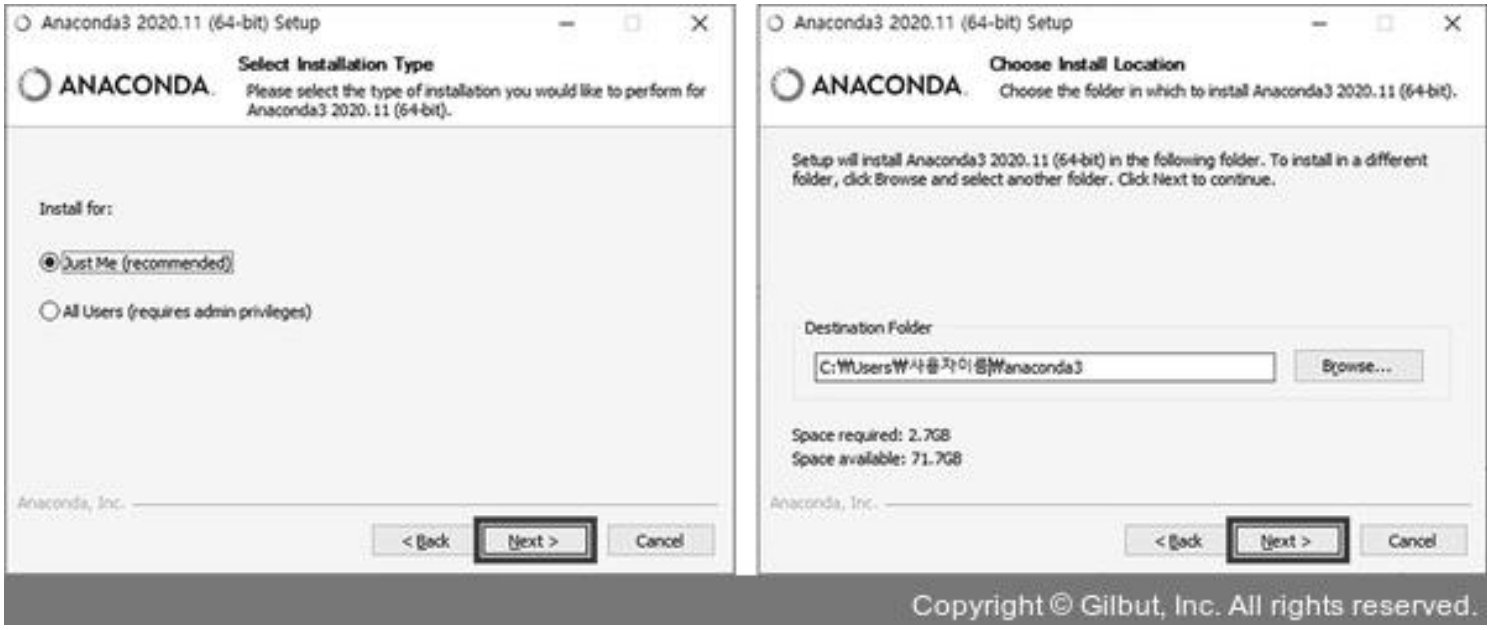
▲ 그림 1-17 운영 체제에 맞는 버전 선택하기

4 내려받은 설치 파일(*.exe)을 클릭해 실행합니다. 설치 프로그램의 초기 화면이 나오면 [Next] 버튼을 클릭하고 라이선스 동의 화면에서는 [I Agree] 버튼을 클릭합니다.



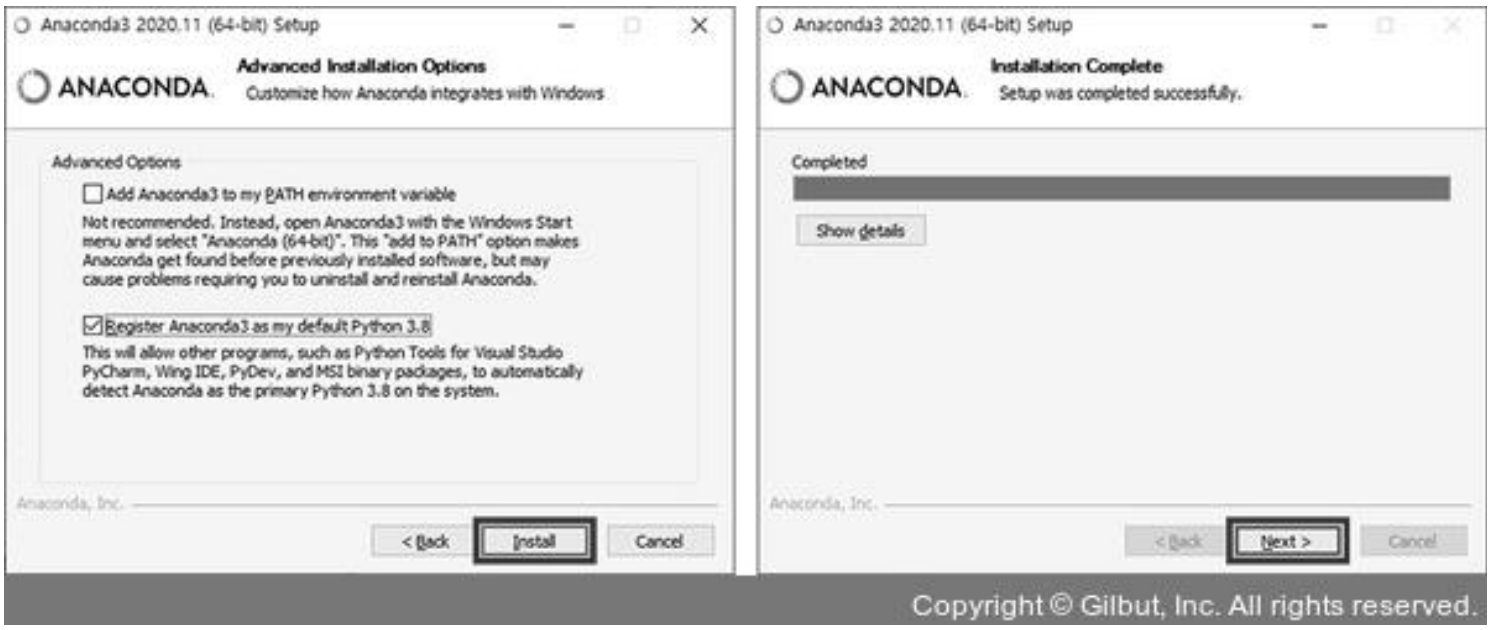
▲ 그림 1-18 아나콘다 설치 시작하고 라이선스 동의하기

5 설치 계정을 선택하는 화면이 나옵니다. 공용 PC라면 Just Me를 고르고 개인 PC라면 All Users를 선택해도 됩니다. 선택한 후 [Next] 버튼을 클릭하면 아나콘다를 설치할 경로가 표시됩니다. 기본값을 그대로 사용해도 되고 변경해도 됩니다. 단, 경로를 바꿀 경우 어디에 설치했는지 설치 경로를 기억해 두세요. 이 책에서는 기본 경로로 설치한다고 가정하고 진행합니다.



▲ 그림 1-19 설치 계정과 설치 경로 선택하기

6 설치 옵션을 선택하는 화면이 나오면 아래쪽 체크박스(Register Anaconda as my default Python)를 선택한 후 [Install] 버튼을 클릭합니다. 위쪽 체크박스는 환경변수를 추가하는 내용인데, 지금은 환경변수를 추가할 필요가 없으므로 넘어갑니다. 설치가 끝나면 [Next] 버튼을 클릭합니다.



▲ 그림 1-20 설치 옵션 선택하고 설치하기

7 PyCharm 안내 화면이 나오면 [Next] 버튼을 클릭합니다. 마지막으로 설치 완료 화면이 나오면 [Finish] 버튼을 클릭합니다.



▲ 그림 1-21 설치 완료하기

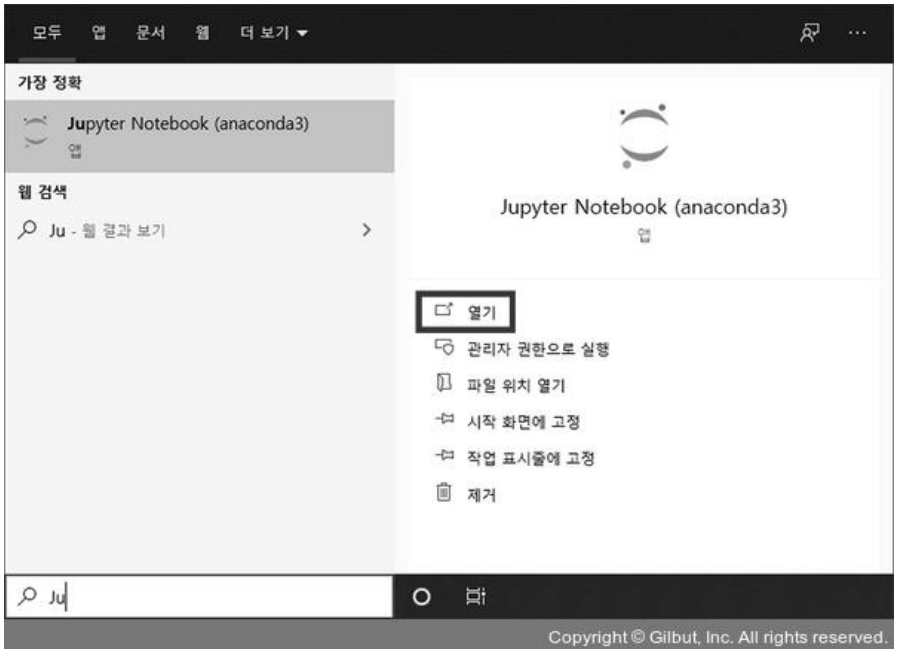
아나콘다가 설치됐습니다. 아나콘다를 설치하면서 주피터 노트북도 함께 설치되므로 별도로 주피터 노트북을 설치하지 않아도 됩니다.

주피터 노트북 사용하기

운영 체제마다 실행 방법이 조금씩 다르므로 운영 체제별로 나눠서 설명합니다.

Windows

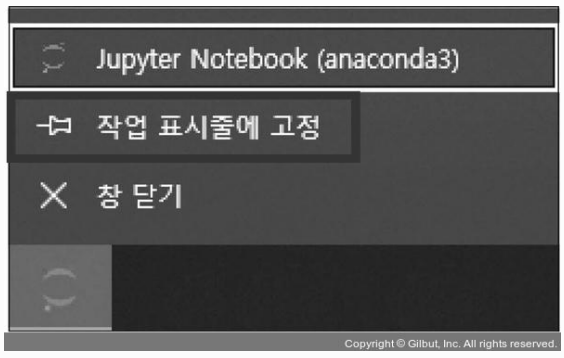
화면 왼쪽 하단 검색창에서 Jupyter Notebook을 검색합니다(Ju만 입력해도 검색 결과에 바로 나옵니다). 검색 결과에 나온 주피터 노트북을 클릭해 실행합니다.



▲ 그림 1-22 Jupyter Notebook 검색해 실행하기

NOTE 주피터 노트북 바로 실행하기

주피터 노트북을 매번 검색해 실행하려면 귀찮죠? 이럴 때 주피터 노트북을 작업 표시줄에 고정하면 매번 검색할 필요 없이 아이콘을 클릭해서 한 번에 주피터 노트북을 실행할 수 있습니다. 주피터 노트북을 실행한 후 화면 하단의 작업 표시줄에서 주피터 노트북 아이콘을 마우스 우클릭합니다. 메뉴에서 [작업 표시줄에 고정]을 클릭하면 작업 표시줄에 주피터 노트북 아이콘이 고정됩니다.



▲ 1-23 주피터 노트북 아이콘을 작업 표시줄에 고정하기

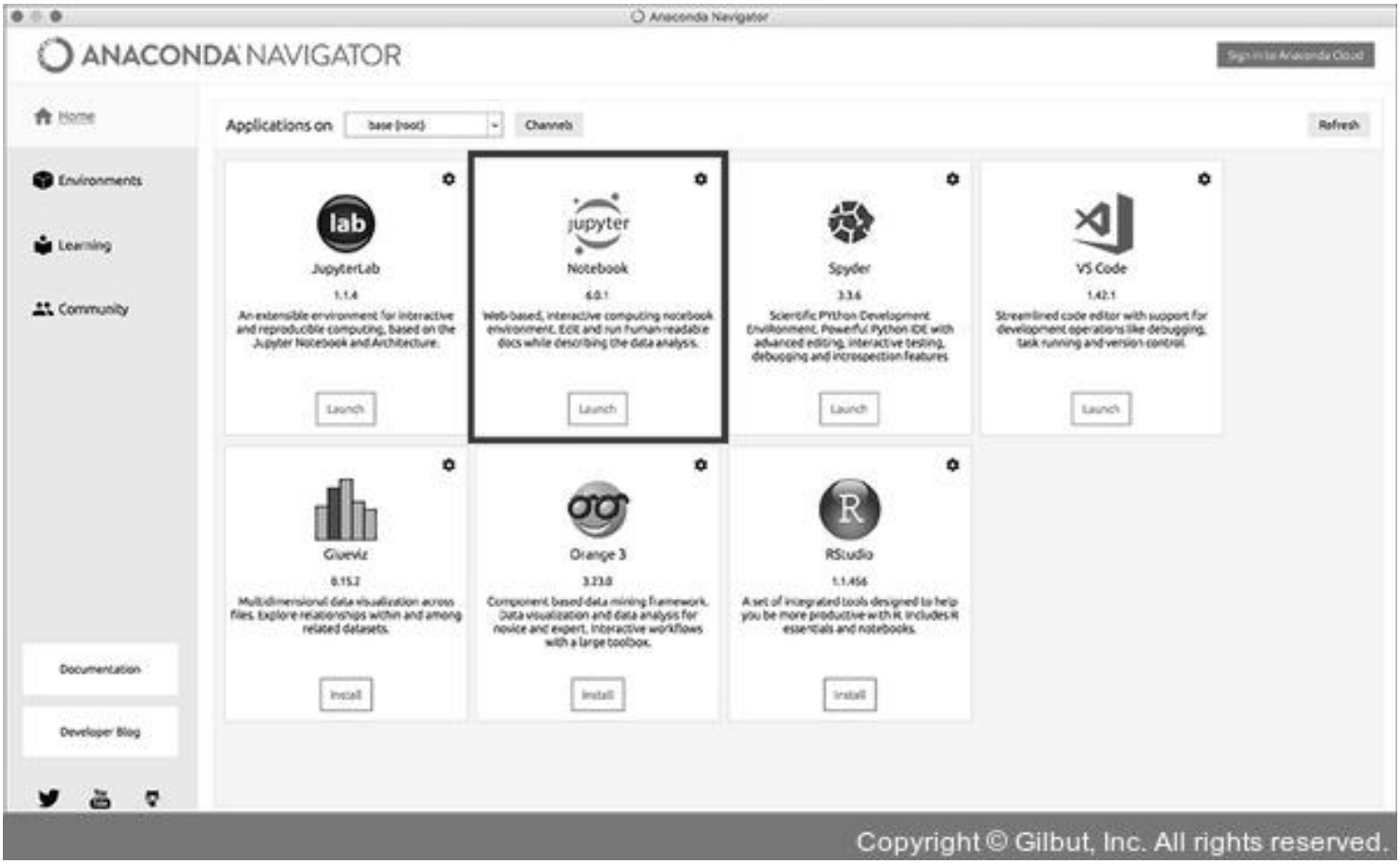
macOS

1 Anaconda Navigator를 엽니다. 방법은 두 가지입니다.

① Cmd + Space bar로 스포트라이트(spotlight) 열기 → anaconda 검색

② launchpad 클릭 → anaconda navigator 클릭

2 메뉴에서 주피터 노트북을 클릭합니다.



▲ 그림 1-24 아나콘다에서 주피터 노트북 실행하기

기본 화면 확인하기

주피터 노트북을 실행하면 다음과 같은 명령 프롬프트 창이 뜹니다. 이 창을 끄면 주피터 노트북이 작동하지 않으므로 주피터 노트북을 실행하는 동안에는 이 창을 반드시 켜 두세요.



▲ 그림 1-25 주피터 노트북 실행 시 활성화되는 명령 프롬프트

조금 기다리면 기본 브라우저에 여러 폴더가 있는 화면이 뜹니다. 이 화면은 주피터 노트북이 설치된 PC의 폴더를 보여 줍니다(여기서는 C:\Users\사용자이름).



▲ 그림 1-26 주피터 노트북 기본 폴더

이때 기본 브라우저로 크롬을 사용해야 프로그래밍하기가 편합니다. 간단한 프로그래밍에서는 큰 문제가 없지만, 8장부터 등장하는 그래프 그리기는 크롬에서 진행해야 결과가 제대로 보입니다.

NOTE 브라우저에서 직접 주피터 노트북 폴더 열기

명령 프롬프트는 뜨는데 브라우저 화면이 뜨지 않는 경우가 가끔 있습니다. 이럴 때는 명령 프롬프트에서 주소를 직접 복사해 브라우저의 주소창에 붙여 넣으면 됩니다. 3개 모두 같은 주소이니 하나를 선택해 사용하세요.



▲ 그림 1-27 폴더 주소 복사하기

기본 브라우저가 크롬이 아니라면 인터넷 익스플로러에서 폴더 화면이 뜹니다. 이때 크롬 브라우저를 열고 주소를 붙여 넣으면 '이 앱을 실행할 때 기본 프로그램으로 설정하시겠습니까?'라는 메시지가 나옵니다. 여기서 [예] 버튼을 클릭하면 크롬을 기본 브라우저로 설정합니다.

브라우저 화면에서 눈에 익는 폴더가 몇 개 보입니다. 여기서 [Desktop]은 바탕화면, [Documents]는 문서, [Downloads]는 다운로드 폴더를 나타냅니다. 파이썬 파일을 만들고 싶은 폴더를 클릭하면 됩니다.

주피터 노트북 사용하기

바탕화면에 파이썬 파일 하나를 만들어 봅시다. 화면 오른쪽에 있는 [New] 버튼을 클릭합니다. 메뉴가 나오면 가장 위에 있는 [Python 3]를 선택합니다. 새 파일이 만들어지고, 만들어진 파일이 브라우저에서 열립니다.



▲ 그림 1-28 새 파일 만들기

주피터 노트북에는 다양한 기능이 있지만 여기에서는 기본으로 알아 두어야 할 기능 세 가지만 짚어 보겠습니다.

새 파일을 보면 상단에 Untitled라고 적힌 부분이 있습니다. 이 부분을 클릭하면 파일명을 수정할 수 있습니다. 원하는 이름을 입력한 후 [Rename] 버튼을 클릭합니다. Untitled 부분이 입력한 이름으로 바뀝니다.



▲ 그림 1-29 파일명 변경하기

주피터 노트북도 코드를 셀 단위로 실행합니다. 셀 안에 명령어를 입력하고 상단 메뉴에서 [Run] 버튼을 클릭하거나 키보드에서 Shift + Enter를 누르면 해당 셀이 실행됩니다. 실행결과는 셀 바로 아래 나타납니다. 아래에 있는 셀을 실행하면 실행결과가 역시 그 아래 나타납니다.

셀을 추가하고 싶을 때는 상단 메뉴에서 [+] 버튼을 클릭합니다. 셀을 제거하려면 [+] 버튼 옆에 있는 가위 모양의 버튼(

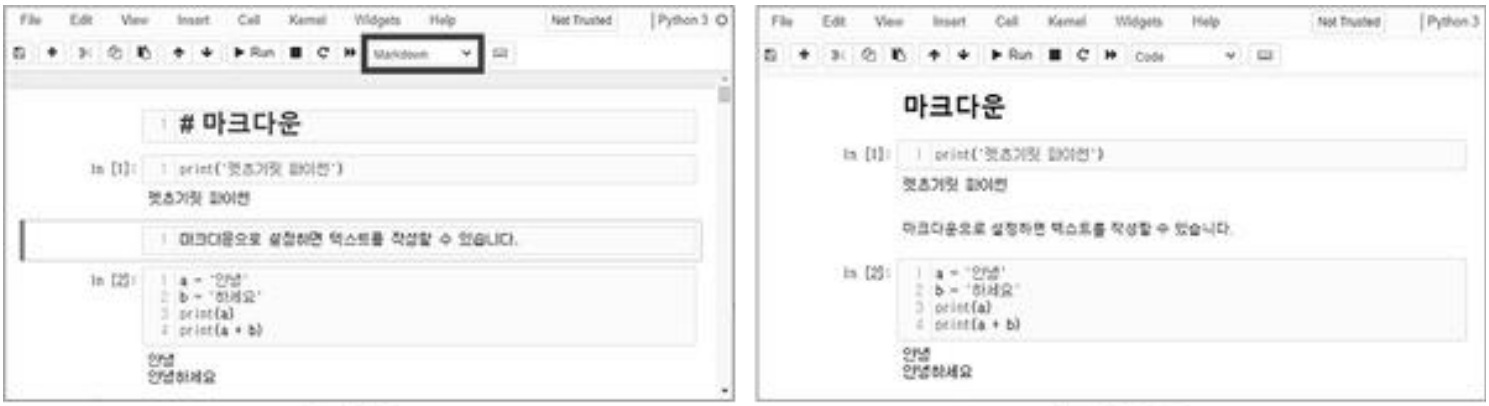
)을 클릭합니다. 파일 내용을 저장하고 싶으면 디스크 모양의 버튼(

)을 클릭하거나 키보드에서 Shift + S 키를 누르면 됩니다.



▲ 그림 1-30 파일 저장하기와 셀 추가/실행/삭제하기

파일 메뉴에서 중간 부분을 보면 드롭 다운으로 선택할 수 있는 메뉴가 있습니다. 셀 유형을 고르는 메뉴입니다. 메뉴를 클릭하면 Code, Markdown, Raw NBConvert, Heading이 있는데, 이 책에서 주로 사용하는 유형은 Code(코드)입니다. Code는 코드를 작성하는 셀로, 실제 파이썬 코드를 작성하고 실행합니다. Markdown(마크다운)은 텍스트를 편집할 수 있는 셀로, 이 책을 학습하면서 필요한 텍스트를 넣고 싶을 때 선택적으로 사용할 수 있습니다. Markdown 셀 역시 상단의 [Run] 버튼이나 Shift + Enter 키로 실행해야 텍스트가 반영됩니다.



Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 1-31 Markdown 셀 실행하기

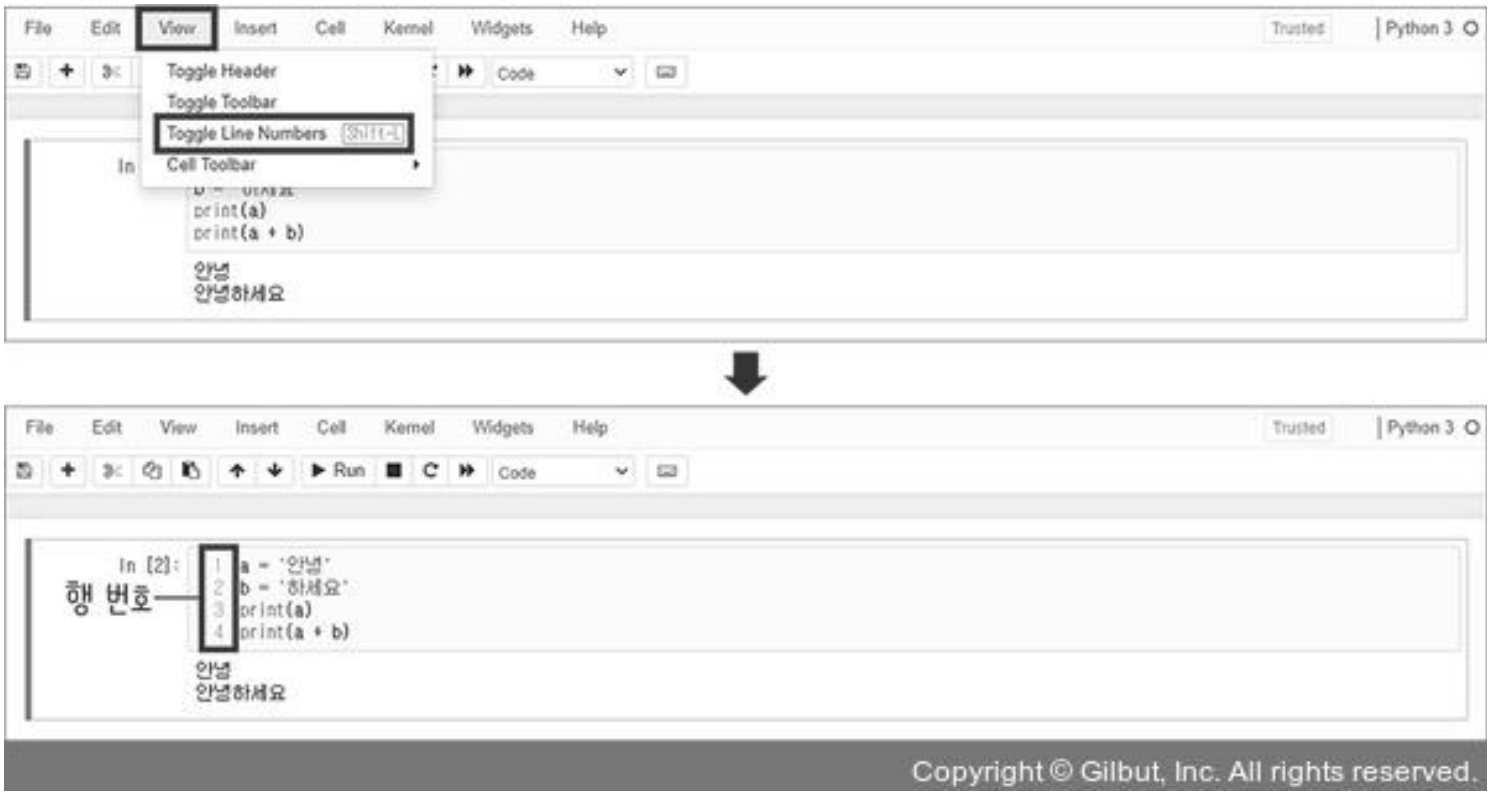
Raw NBConvert는 발표용 슬라이드를 만들 때 사용하는 셀로 이 책에서는 다루지 않습니다. Heading은 제목을 나타내는 셀입니다. Heading을 클릭하면 '주피터 노트북에서는 더이상 Heading 셀을 제공하지 않는다. 그 대신 Markdown 셀에서 #을 사용해 Heading을 사용할 수 있다.'라는 알림 창이 뜹니다. 알림 창 내용처럼 Markdown 셀에서 #을 사용해 파이썬 파일 안에 제목(헤더)을 붙일 수 있습니다. 사용할 수 있는 최대 개수는 4개로 #을 많이 사용할수록 작은 크기의 제목이 만들어집니다.



Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 1-32 Markdown 셀에 제목 넣기

코랩처럼 주피터 노트북의 코드 셀에도 행 번호가 표시되게 해 봅시다. 메뉴에서 [View → Toggle Line Numbers]를 선택하면 코드 셀에 행 번호가 나타납니다.



▲ 그림 1-33 코드 셀에 행 번호 추가하기

코랩과 주피터 노트북 중 본인의 환경에 더 맞는 에디터를 선택해 테스트 파일을 만들어 보세요. 이 책에서는 코랩을 기본 에디터로 사용하고 필요에 따라 주피터 노트북과 혼용합니다.

이로써 파이썬으로 프로그래밍하기 위한 준비가 모두 끝났습니다. 그럼 다음 장부터 본격적으로 파이썬을 배워 봅시다.